

```
import pandas as pd
import seaborn as sns
df = sns.load_dataset('titanic')
print(df.head())
print(df.shape)
df.info()
```

```
In [42]: df.loc[0, 'embark_town'] = 'southampton'
df.loc[1, 'embark_town'] = 'SOUTHAMPTON'
df.loc[2, 'embark_town'] = 'southampton'
```

```
In [43]: print(df.loc[0:2, 'embark_town'])

0    southampton
1    SOUTHAMPTON
2    southampton
Name: embark_town, dtype: object
```

```
In [44]: df['embark_town'] = df['embark_town'].str.title()
```

```
In [45]: print(df.loc[0:2, 'embark_town'])

0    Southampton
1    Southampton
2    Southampton
Name: embark_town, dtype: object
```

```
In [46]: df['embark_town'] = df['embark_town'].str.title()

# Show unique values
print(df['embark_town'].unique())

# Count null values
print(df['embark_town'].isnull().sum())

['Southampton' 'Queenstown' 'Cherbourg' nan]
2
```

```
In [47]: duplicate_rows = df.iloc[:5]
df_dup = pd.concat([df, duplicate_rows])
print(df_dup.shape)

(896, 15)
```

```
In [48]: print(df_dup.duplicated().sum())

112
```

```
In [49]: df_dup[df_dup.duplicated()]
```

```
Out[49]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
47	1	3	female	NaN	0	0	7.7500	Q	Third	woman
76	0	3	male	NaN	0	0	7.8958	S	Third	man
77	0	3	male	NaN	0	0	8.0500	S	Third	man
87	0	3	male	NaN	0	0	8.0500	S	Third	man
95	0	3	male	NaN	0	0	8.0500	S	Third	man
...	...	...	...	...	...	...	...	...	...	...
0	0	3	male	22.0	1	0	7.2500	S	Third	man
1	1	1	female	38.0	1	0	71.2833	C	First	woman
2	1	3	female	26.0	0	0	7.9250	S	Third	woman
3	1	1	female	35.0	1	0	53.1000	S	First	woman
4	0	3	male	35.0	0	0	8.0500	S	Third	man

112 rows × 15 columns

```
In [50]: df_dup = df_dup.drop_duplicates()
print(df_dup.duplicated().sum())
```

0

```
In [51]: df.loc[0, 'embark_town'] = 'Southampton'
df.loc[1, 'embark_town'] = 'Southampton'
```

```
In [52]: print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    645
Cherbourg      167
Queenstown      77
Name: count, dtype: int64
```

```
In [53]: df.loc[0, 'age'] = 240
df.loc[1, 'age'] = 300
```

```
In [54]: df['age'].describe()
```

```
Out[54]: count    714.000000
mean      30.371387
std       19.362012
min        0.420000
25%       20.125000
50%       28.000000
75%       38.750000
max       300.000000
Name: age, dtype: float64
```

```
In [55]: df.loc[df['age'] > 120, 'age'] = (
df.loc[df['age'] > 120, 'age'] / 12
```

```
)
df['age'].describe()
```

```
Out[55]: count      714.000000
         mean       29.678109
         std        14.525902
         min         0.420000
         25%        20.000000
         50%        28.000000
         75%        38.000000
         max        80.000000
         Name: age, dtype: float64
```

Wrong Category Labels

```
In [56]: df['survived_label'] = df['survived'].map({
         0: 'No',
         1: 'Yes'
       })
         print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	20.0	1	0	7.2500	S	Third
1	1	1	female	25.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	No
1	woman	False	C	Southampton	yes	False	Yes
2	woman	False	NaN	Southampton	yes	True	Yes
3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

Redundant Columns

```
In [57]: df.isnull().mean()
```

```
Out[57]: survived      0.000000
         pclass        0.000000
         sex           0.000000
         age           0.198653
         sibsp         0.000000
         parch         0.000000
         fare          0.000000
         embarked      0.002245
         class         0.000000
         who           0.000000
         adult_male     0.000000
         deck          0.772166
         embark_town    0.002245
         alive          0.000000
         alone          0.000000
         survived_label 0.000000
         dtype: float64
```

```
In [58]: df.drop(columns=['deck'], inplace=True)
df.head()
```

```
Out[58]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
0	0	3	male	20.0	1	0	7.2500	S	Third	man
1	1	1	female	25.0	1	0	71.2833	C	First	woman
2	1	3	female	26.0	0	0	7.9250	S	Third	woman
3	1	1	female	35.0	1	0	53.1000	S	First	woman
4	0	3	male	35.0	0	0	8.0500	S	Third	man

```
In [59]: df.isnull().sum()
```

```
Out[59]: survived      0
pclass      0
sex         0
age        177
sibsp      0
parch      0
fare       0
embarked    2
class      0
who        0
adult_male  0
embark_town 2
alive      0
alone      0
survived_label  0
dtype: int64
```

```
In [64]: df['age'] = df['age'].fillna(df['age'].mean())
df['age'] = df['age'].astype(int)
df.head(20)
```

Out[64]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	wh
0	0	3	male	20	1	0	7.2500	S	Third	ma
1	1	1	female	25	1	0	71.2833	C	First	woma
2	1	3	female	26	0	0	7.9250	S	Third	woma
3	1	1	female	35	1	0	53.1000	S	First	woma
4	0	3	male	35	0	0	8.0500	S	Third	ma
5	0	3	male	29	0	0	8.4583	Q	Third	ma
6	0	1	male	54	0	0	51.8625	S	First	ma
7	0	3	male	2	3	1	21.0750	S	Third	chi
8	1	3	female	27	0	2	11.1333	S	Third	woma
9	1	2	female	14	1	0	30.0708	C	Second	chi
10	1	3	female	4	1	1	16.7000	S	Third	chi
11	1	1	female	58	0	0	26.5500	S	First	woma
12	0	3	male	20	0	0	8.0500	S	Third	ma
13	0	3	male	39	1	5	31.2750	S	Third	ma
14	0	3	female	14	0	0	7.8542	S	Third	chi
15	1	2	female	55	0	0	16.0000	S	Second	woma
16	0	3	male	2	4	1	29.1250	Q	Third	chi
17	1	2	male	29	0	0	13.0000	S	Second	ma
18	0	3	female	31	1	0	18.0000	S	Third	woma
19	1	3	female	29	0	0	7.2250	C	Third	woma

In [65]: `print(df['embarked'].isnull().sum())`

2

In [66]: `df['embarked_ffill'] = df['embarked'].ffill()
print(df['embarked_ffill'].isnull().sum())`

0

KNN Imputation

In [67]: `import pandas as pd
import numpy as np
data = {
 'age': [22,25, np.nan, 24, 23],
 'fare': [7.25,8.05,7.90,8.00,7.50]
}
df = pd.DataFrame(data)
print(df)`

	age	fare
0	22.0	7.25
1	25.0	8.05
2	NaN	7.90
3	24.0	8.00
4	23.0	7.50

```
In [68]: from sklearn.impute import KNNImputer
KNN = KNNImputer(n_neighbors=3)
df_KNN = pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns
)
print(df_KNN)
```

	age	fare
0	22.0	7.25
1	25.0	8.05
2	24.0	7.90
3	24.0	8.00
4	23.0	7.50

```
In [70]: import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
df = sns.load_dataset('titanic')
df['embarked_ffill'] = df['embarked'].ffill()
cols = [
    'age',
    'fare',
    'sibsp',
    'parch',
    'pclass'
]
imputer = KNNImputer(n_neighbors = 5)
df[cols] = imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0
embarked_ffill	0
dtype:	int64

Backward fill `df.fillna(method='bfill',inplace=True)` or `df.bfill(inplace=True)`

Encoding Using Titanic Dataset

what is Encoding?

Machine Learning models work with numbers, not text. example: male female male
A model cannot directly understand "male" and "Female". We converted categories into number using Encoding.

Types of Encoding

1.Label Encoding(Ordinal Data) Assign a number to each category Example: size . small . medium . Large becomes size . 0 . 1 . 2 . The order matters.

When to use Label Encoding ordinal Categories example: small < medium < large

Nominal Categories

Example : red, blue, green Male, Female India, USA, UK

2. One-Hot Encoding(Nominal Data) crates a separate binary column for each category.

When to use One-Hot Encoding?

Nominal categories Example: country,Gender,color,city,department

3.Target Encoding Replace categories with the average value of target variable.

Why Use Target Encoding? imagine: 5000 cities one-Hot Encoding would create: 5000 columns Not practical Target Encoding

```
In [2]: import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
df = sns.load_dataset('titanic')
#Label Encoding
le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(df['sex'])
df = pd.get_dummies(df,columns=['embarked'])
target_map = df.groupby('pclass')['survived'].mean()
df['class_target'] = (df['pclass'].map(target_map))
df.head()
```

```
Out[2]:
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male
0	0	3	male	22.0	1	0	7.2500	Third	man	True
1	1	1	female	38.0	1	0	71.2833	First	woman	False
2	1	3	female	26.0	0	0	7.9250	Third	woman	False
3	1	1	female	35.0	1	0	53.1000	First	woman	False
4	0	3	male	35.0	0	0	8.0500	Third	man	True

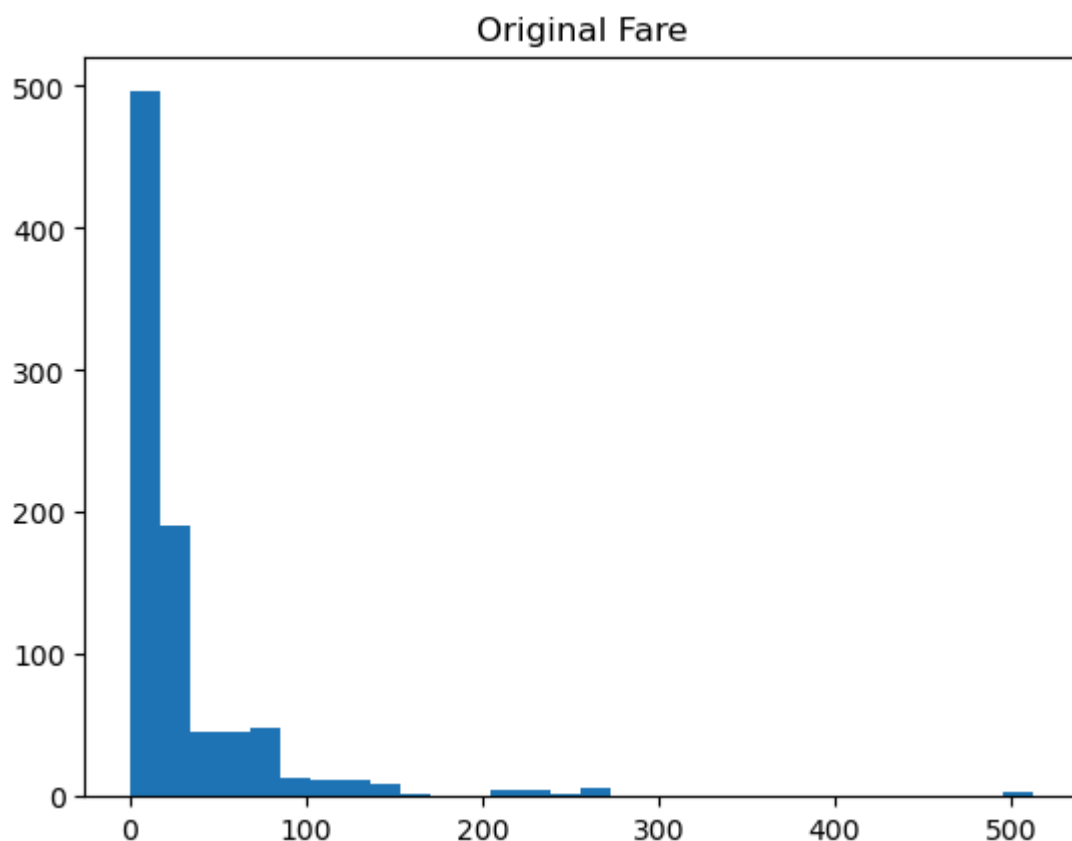
Feature Scaling

```
In [3]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['age_minmax', 'fare_minmax']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_minmax', 'fare_minmax']].head()
```

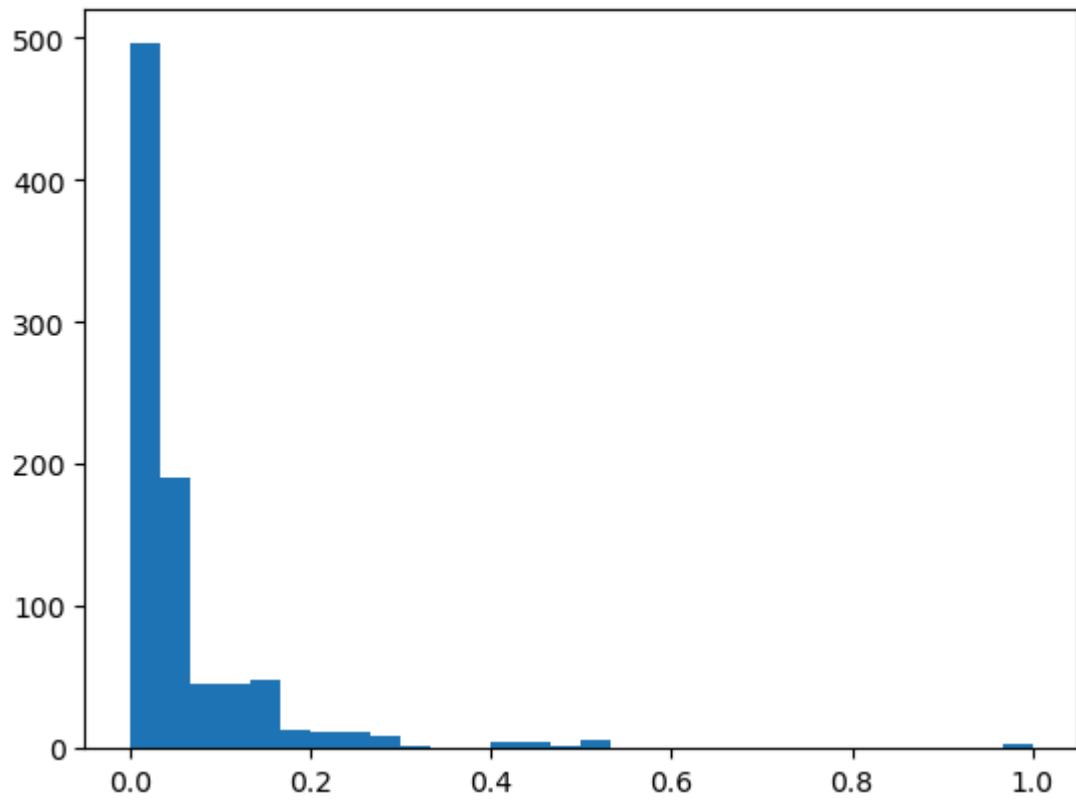
```
Out[3]:
```

	age_minmax	fare_minmax
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

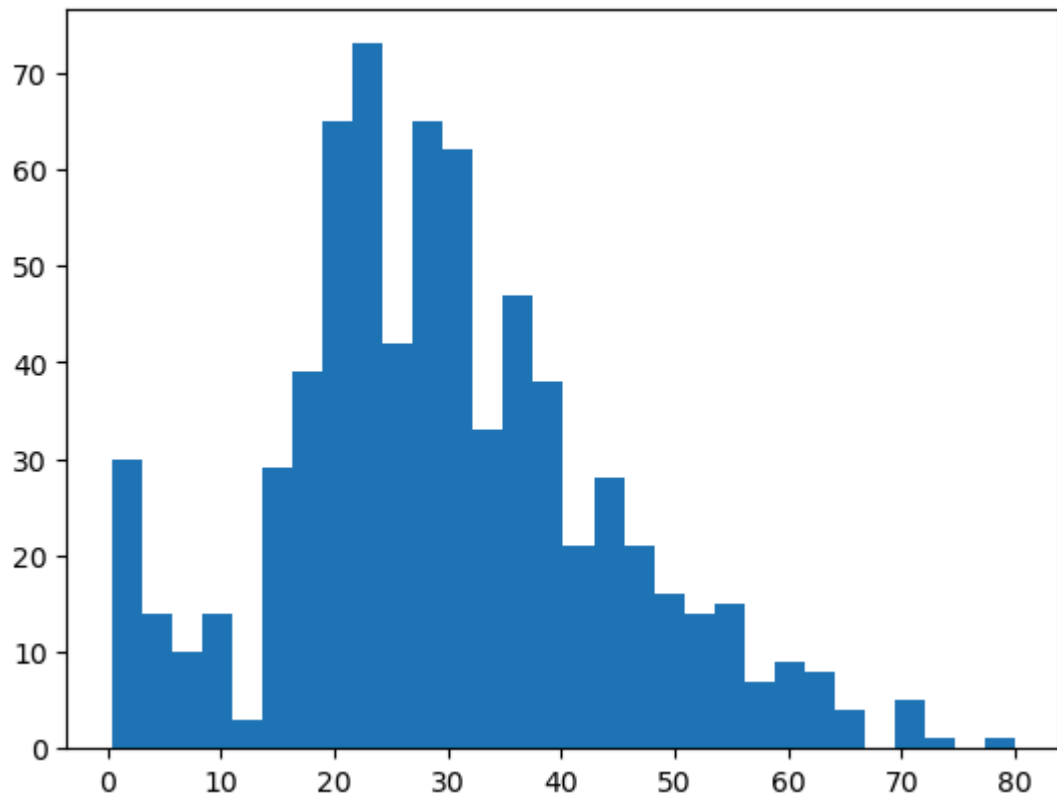
```
In [5]: import matplotlib.pyplot as plt
plt.hist(df['fare'].dropna(), bins=30)
plt.title("Original Fare")
plt.show()
plt.hist(df['fare_minmax'], bins=30)
plt.title("MinMax Scaled Fare")
plt.show()
plt.hist(df['age'].dropna(), bins=30)
plt.title("Original Age")
plt.show()
plt.hist(df['age_minmax'], bins=30)
plt.title("MinMax Scaled Age")
plt.show()
```

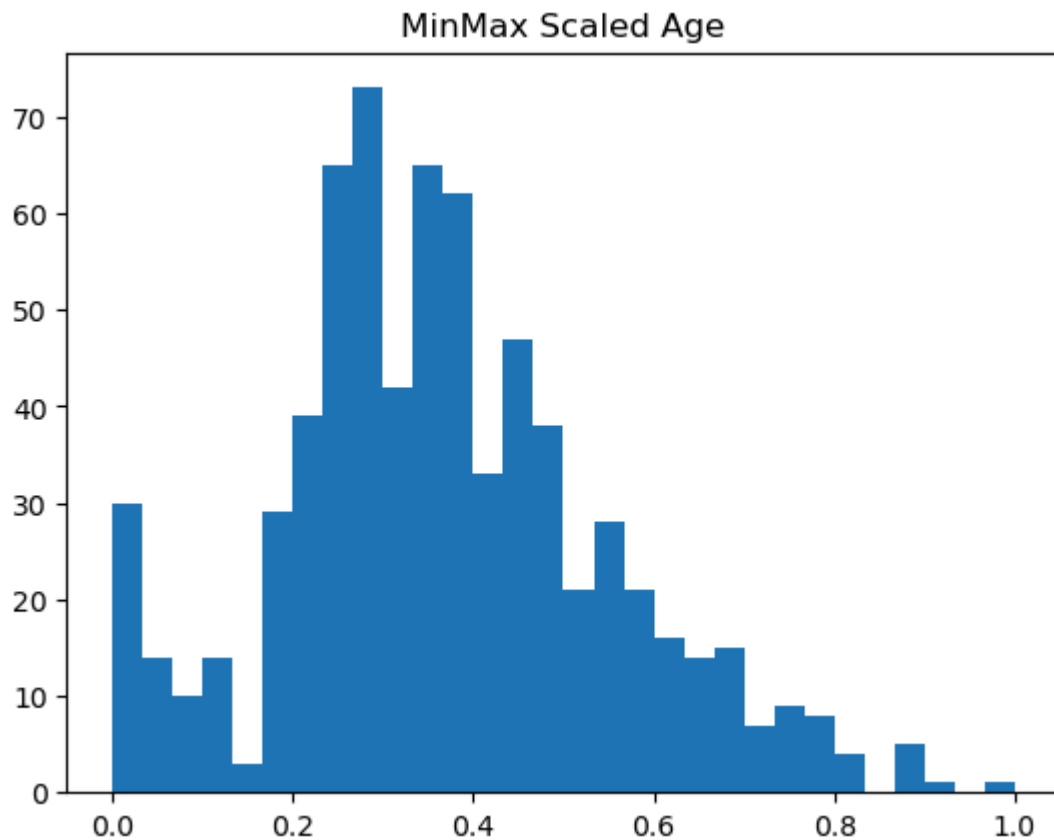


MinMax Scaled Fare



Original Age





```
In [8]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['age_std', 'fare_std']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_std', 'fare_std']].head()
```

```
Out[8]:
```

	age_std	fare_std
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

```
In [7]: from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df[['age_std', 'fare_std']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_std', 'fare_std']].head()
```

Out[7]:

	age_std	fare_std
0	-0.335664	-0.312011
1	0.559441	2.461242
2	-0.111888	-0.282777
3	0.391608	1.673732
4	0.391608	-0.277363

In []: